ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №7

«Робота з базами даних у Python»

з курсу «Професійний Python»
Варіант №13 — «Система обліку комп'ютерної техніки»

Виконав: Капарник Дмитро Андрійович

Група: ФЕП-32

Перевірив: доцент Куньо Іван Михайлович

Дата: 18.10.2026

Львів — 2026

1. Мета роботи

Додати persistence layer на SQLite, освоїти SQL, DB-API, ORM, транзакції та міграції.

2. Індивідуальний варіант і постановка задачі

Варіант №13 — система обліку комп'ютерної техніки. Створити manufacturers і devices з UNIQUE inventory_number, ManufacturerRepository та DeviceRepository. Зберегти базові операції, додати CRUD і зміну ціни; пакетна реєстрація повинна мати rollback при дублікаті. Міграція додає location.

Наскрізний проєкт продовжує ЛР №1. Базові поля: інвентарний номер, тип, виробник, модель і вартість. Збережено пошук, фільтр виробника, середню/загальну вартість і визначення найдорожчого пристрою.

3. Структура проєкту та створені модулі

```
inventory-service/
  pyproject.toml, README.md, .gitignore
  src/computer_inventory/
    models.py
    database.py
    services.py
  src/inventory_service/  # фінальний пакет запуску
  tests/
  scripts/
  docs/GITHUB.md
```

4. Схема бази даних

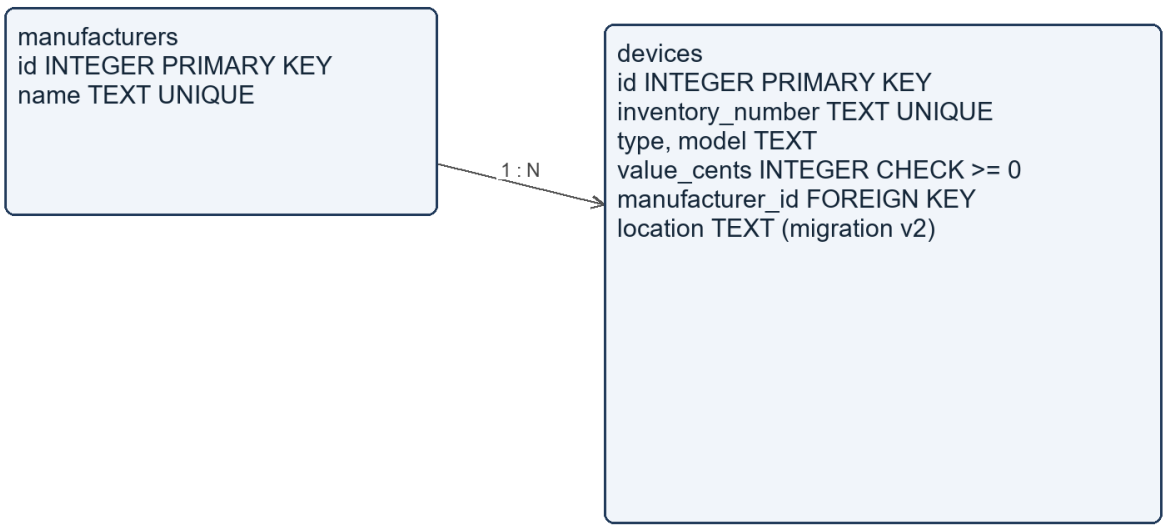


Рисунок 1 — Зв’язок manufacturers 1:N devices

Поле	Тип / обмеження	Призначення
manufacturers.id	INTEGER PRIMARY KEY	Ідентифікатор виробника
manufacturers.name	TEXT UNIQUE, NOCASE	Назва виробника
devices.id	INTEGER PRIMARY KEY	Ідентифікатор пристрою
inventory_number	TEXT UNIQUE, NOCASE	Унікальний інвентарний код
type, model	TEXT NOT NULL	Опис техніки
value_cents	INTEGER CHECK >= 0	Точна вартість у копійках
manufacturer_id	FOREIGN KEY → manufacturers.id	Зв’язок виробника
location	TEXT NULL	Додається міграцією v2

```
CREATE TABLE manufacturers (id INTEGER PRIMARY KEY, name TEXT COLLATE NOCASE NOT NULL
UNIQUE);
CREATE TABLE devices (id INTEGER PRIMARY KEY, inventory_number TEXT COLLATE NOCASE NOT
NULL UNIQUE,
 type TEXT NOT NULL, model TEXT NOT NULL, value_cents INTEGER NOT NULL CHECK(value_cents
>= 0),
 manufacturer_id INTEGER NOT NULL REFERENCES manufacturers(id));
PRAGMA user_version=1;
```

5. ORM, з’єднання та репозиторії

```
class ManufacturerRow(Base):
    __tablename__ = "manufacturers"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(collation="NOCASE"), unique=True)
    devices: Mapped[list["DeviceRow"]] = relationship(back_populates="manufacturer")

class DeviceRow(Base):
    __tablename__ = "devices"
    __table_args__ = (CheckConstraint("value_cents >= 0"),)
    id: Mapped[int] = mapped_column(primary_key=True)
    inventory_number: Mapped[str] = mapped_column(
        String(collation="NOCASE"), unique=True
    )
    type: Mapped[str] = mapped_column(String)
    model: Mapped[str] = mapped_column(String)
    value_cents: Mapped[int] = mapped_column(Integer)
    manufacturer_id: Mapped[int] = mapped_column(ForeignKey("manufacturers.id"))
    manufacturer: Mapped[ManufacturerRow] = relationship(back_populates="devices")
    location: Mapped[str | None] = mapped_column(String, nullable=True)

def make_engine(url: str) -> Engine:
    engine = create_engine(url, connect_args={"check_same_thread": False})

    @event.listens_for(engine, "connect")
    def enable_foreign_keys(connection: Any, record: Any) -> None:
        connection.execute("PRAGMA foreign_keys=ON")
```

```
    raw = engine.raw_connection()
    try:
        connection = raw.driver_connection
        assert isinstance(connection, sqlite3.Connection)
        migrate(connection)
    finally:
        raw.close()
    return engine

class ManufacturerRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    def add(self, name: str) -> ManufacturerRow:
        if not name.strip():
            raise ValueError("Manufacturer cannot be empty")
        row = ManufacturerRow(name=name.strip())
        self.session.add(row)
        self.session.flush()
        return row

    def get_or_create(self, name: str) -> ManufacturerRow:
        row = self.session.scalar(
            select(ManufacturerRow).where(
                func.lower(ManufacturerRow.name) == name.strip().lower()
            )
        )
        return row if row is not None else self.add(name)

    def all(self) -> list[ManufacturerRow]:
        return list(
            self.session.scalars(select(ManufacturerRow).order_by(ManufacturerRow.id))
        )
```

Підключення вмикає PRAGMA foreign_keys=ON. SQLAlchemy Session обмежує час життя операцій; DeviceRepository перетворює Decimal-ціну на цілу кількість копійок. Repository приховує SQL від сервісу й API. Точний пошук номера використовує UNIQUE-індекс із NOCASE.

```
class DeviceRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    def add(self, device: Device, location: str | None = None) -> DeviceRow:
        manufacturer = ManufacturerRepository(self.session).get_or_create(
            device.manufacturer
        )
        row = DeviceRow(
            inventory_number=device.inventory_number,
            type=device.device_type,
            model=device.model,
            value_cents=int(device.cost * 100),
            manufacturer_id=manufacturer.id,
            location=location,
        )
```

```
    self.session.add(row)
    self.session.flush()
    return row

def get(self, device_id: int) -> DeviceRow | None:
    return self.session.get(DeviceRow, device_id)

def find(self, inventory_number: str) -> DeviceRow | None:
    return self.session.scalar(
        select(DeviceRow).where(
            DeviceRow.inventory_number == inventory_number.strip()
        )
    )

def all(self, manufacturer: str | None = None) -> list[DeviceRow]:
    statement = select(DeviceRow).join(DeviceRow.manufacturer)
    if manufacturer:
        statement = statement.where(
            func.lower(ManufacturerRow.name) == manufacturer.strip().lower()
        )
    return list(self.session.scalars(statement.order_by(DeviceRow.id)))

def change_cost(self, device_id: int, cost: Decimal) -> DeviceRow:
    row = self.get(device_id)
    if row is None:
        raise LookupError(device_id)
    # Reuse the domain validation, including cents and upper bound.
    Device(row.inventory_number, row.type, row.manufacturer.name, row.model, cost)
    row.value_cents = int(cost * 100)
    self.session.flush()
    return row

def delete(self, device_id: int) -> None:
    row = self.get(device_id)
    if row is None:
        raise LookupError(device_id)
    self.session.delete(row)
    self.session.flush()

def statistics(self) -> dict[str, Any]:
    count, total = self.session.execute(
        select(
            func.count(DeviceRow.id),
            func.coalesce(func.sum(DeviceRow.value_cents), 0),
        )
    ).one()
    best = self.session.scalar(
        select(DeviceRow)
        .order_by(DeviceRow.value_cents.desc(), DeviceRow.id)
        .limit(1)
    )
    return {
        "count": count,
        "total": f"{Decimal(total) / 100:.2f}",
        "average": str(
            (Decimal(total) / 100 / count).quantize(
```

```
                    Decimal("0.01"), rounding=ROUND_HALF_UP
                )
            )
            if count
            else "0.00",
            "most_expensive_id": best.id if best else None,
        }
```

6. DB-API, CRUD і транзакція

```
def dbapi_search(
    path: Path, inventory_number: str, manufacturer: str = ""
) -> list[tuple[Any, ...]]:
    with sqlite3.connect(path) as connection:
        return connection.execute(
            "SELECT d.inventory_number, m.name, d.value_cents FROM devices d "
            "JOIN manufacturers m ON m.id=d.manufacturer_id "
            "WHERE d.inventory_number = ? COLLATE NOCASE "
            "AND (? = '' OR m.name = ? COLLATE NOCASE)",
            (inventory_number, manufacturer, manufacturer),
        ).fetchall()

def register_batch(session: Session, devices: list[Device]) -> None:
    """Caller provides a fresh session; all inserts commit or all roll back."""
    with session.begin():
        repository = DeviceRepository(session)
        for device in devices:
            repository.add(device)
```

Значення DB-API передаються окремо через ?. Дані на зразок x' OR 1=1 -- залишаються значенням, а не SQL-кодом. add/get/change_cost/delete реалізують CRUD. register_batch отримує нову Session і відкриває session.begin: при успіху зміни commit, при IntegrityError — rollback усієї групи. Для іншого життєвого циклу транзакцією має керувати викликач.

ACID	**Реалізація**
Atomicity	Одна транзакція на пакет, повний rollback при дублікаті
Consistency	PRIMARY KEY, UNIQUE, FOREIGN KEY, CHECK та доменна валідація
Isolation	Транзакції та блокування SQLite; записи серіалізуються
Durability	Підтверджені зміни записуються SQLite з гарантіями обраних режимів сховища

7. Міграція location та результати

```
def migrate(connection: sqlite3.Connection) -> None:
    """Upgrade v0 -> v1 -> v2 once; user_version records the schema version."""
    version = connection.execute("PRAGMA user_version").fetchone()[0]
```

```
    if version > 2:
        raise ValueError("Database schema is newer than this application")
    with connection:
        if version < 1:
            connection.executescript(SCHEMA_V1)
            connection.execute("PRAGMA user_version=1")
        if version < 2:
            connection.execute("ALTER TABLE devices ADD COLUMN location TEXT")
            connection.execute("PRAGMA user_version=2")


-- Upgrade, performed once by database.migrate.
ALTER TABLE devices ADD COLUMN location TEXT;
PRAGMA user_version=2;
-- SQLite 3.35+ downgrade (requires a separate controlled migration):
-- ALTER TABLE devices DROP COLUMN location;
-- PRAGMA user_version=1;
```

Версію схеми збережено у PRAGMA user_version. v1 створює дві таблиці, v2 виконує ALTER TABLE і додає nullable location без втрати попередніх рядків. Повторне відкриття v2 не повторює ALTER. Новіша за підтримувану версія відхиляється. Використано власну просту версійовану міграцію; Alembic є масштабованим розвитком цього механізму.

```
{
  "count": 4,
  "total": "92700.00",
  "average": "23175.00",
  "most_expensive_id": 3
}
Dell: ['INV-002', 'INV-003']
Duplicate batch rolled back
DB-API: [('INV-002', 'Dell', 850000)]
```

8. Integration tests

```
def test_crud_filter_aggregates(engine, devices):
    with Session(engine) as session, session.begin():
        repo = DeviceRepository(session)
        rows = [repo.add(d) for d in devices]
        assert repo.get(rows[0].id).model == devices[0].model
        assert repo.find("inv-001").id == rows[0].id
        assert len(repo.all("Dell")) == 2
        assert repo.statistics()["total"] == "92700.00"
        assert repo.statistics()["average"] == "23175.00"
        repo.change_cost(rows[0].id, Decimal("10000"))
        assert repo.get(rows[0].id).value_cents == 1000000
        repo.delete(rows[-1].id)
        assert len(repo.all()) == 3
        assert len(ManufacturerRepository(session).all()) == 3

def test_unique_batch_rollback(engine, devices):
    with Session(engine) as session:
        with pytest.raises(IntegrityError):
            register_batch(session, [devices[0], devices[0]])
```

```
    with Session(engine) as session:
        assert DeviceRepository(session).all() == []
        assert ManufacturerRepository(session).all() == []

def test_persistent_commit_dbapi(engine, devices):
    with Session(engine) as session:
        register_batch(session, devices)
    path = Path(engine.url.database)
    assert dbapi_search(path, "INV-002", "Dell")[0][2] == 850000
    assert dbapi_search(path, "x' OR 1=1 --") == []

def test_migration_and_foreign_key(engine):
    with engine.connect() as connection:
        assert connection.exec_driver_sql("PRAGMA user_version").scalar() == 2
        assert "location" in [
            row[1] for row in connection.exec_driver_sql("PRAGMA table_info(devices)")
        ]
        assert connection.exec_driver_sql("PRAGMA foreign_keys").scalar() == 1
    with Session(engine) as session:
        with pytest.raises(IntegrityError):
            session.execute(
                text(
                    "INSERT INTO
devices(inventory_number,type,model,value_cents,manufacturer_id) VALUES
('A','PC','M',0,999)"
                )
            )
        session.rollback()
```

Використано окремі temporary SQLite-файли. Перевірено CRUD, manufacturer filter, суму/середнє, UNIQUE, FOREIGN KEY, DB-API, підтвердження даних між Session і повний rollback із видаленням також новоствореного виробника. Перевірки БД та API завершилися успішно; повний suite наведений у підсумковому звіті ЛР10.

```
============================= test session starts ==============================
platform win32 -- Python 3.12.14, pytest-9.1.1, pluggy-1.7.0 --
C:\Users\user\Documents\Codex\2026-10-18\new-chat-2\outputs\computer-
inventory\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\user\Documents\Codex\2026-10-18\new-chat-2\outputs\computer-inventory
configfile: pyproject.toml
plugins: anyio-4.15.1, cov-7.1.0
collecting ... collected 14 items

tests/test_database.py::test_crud_filter_aggregates PASSED               [  7%]
tests/test_database.py::test_unique_batch_rollback PASSED                [ 14%]
tests/test_database.py::test_persistent_commit_dbapi PASSED              [ 21%]
tests/test_database.py::test_migration_and_foreign_key PASSED            [ 28%]
tests/test_database.py::test_empty_and_missing PASSED                    [ 35%]
tests/test_api.py::test_api_crud_statistics PASSED                       [ 42%]
tests/test_api.py::test_api_invalid_value[-1] PASSED                     [ 50%]
tests/test_api.py::test_api_invalid_value[NaN] PASSED                    [ 57%]
tests/test_api.py::test_api_invalid_value[1.001] PASSED                  [ 64%]
tests/test_api.py::test_api_invalid_value[1000000000001] PASSED          [ 71%]
tests/test_api.py::test_api_missing_and_manufacturers PASSED             [ 78%]
```

```
tests/test_api.py::test_async_retry_concurrency PASSED                   [ 85%]
tests/test_api.py::test_async_failure_timeout PASSED                     [ 92%]
tests/test_api.py::test_async_endpoint PASSED                            [100%]

============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  C:\Users\user\Documents\Codex\2026-10-18\new-chat-2\outputs\computer-
inventory\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning:
Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== 14 passed, 1 warning in 1.92s ========================
```

Код і історія лабораторних: https://github.com/dimakaparnyk/professional-python-labs. Для відповідного етапу використовується тег lab-07, фінальна інтегрована реалізація — main (перевірений CI commit 2e3fb3d).

9. Відповіді на контрольні запитання

1. Що таке база даних?

Організоване сховище даних із засобами пошуку, зміни та забезпечення цілісності.

2. Що таке реляційна база даних?

База, що подає дані таблицями зі зв’язками та обмеженнями.

3. Що таке таблиця?

Набір рядків із визначеними колонками та їхніми типами.

4. Що таке рядок таблиці?

Окремий запис сутності, наприклад одного пристрою.

5. Що таке поле?

Колонка або конкретне значення атрибута запису залежно від контексту.

6. Що таке SQL?

Мова визначення, читання та зміни реляційних даних.

7. Для чого використовується CREATE TABLE?

Для створення таблиці з полями, типами й обмеженнями.

8. Що означає CRUD?

Create, Read, Update, Delete — створення, читання, зміна та видалення.

9. Яка SQL-команда відповідає Create?

INSERT додає новий рядок.

10. Яка команда відповідає Read?

SELECT читає дані.

11. Яка команда відповідає Update?

UPDATE змінює наявні рядки.

12. Яка команда відповідає Delete?

DELETE видаляє рядки.

13. Що таке SQLite?

Вбудована реляційна СУБД, яка зазвичай зберігає базу в одному файлі.

14. Які переваги SQLite?

Не потребує окремого сервера, проста в налаштуванні та придатна для навчання й локальних застосунків.

15. Що таке Python DB-API?

Стандартизований інтерфейс взаємодії Python із реляційними драйверами, описаний PEP 249.

16. Для чого використовується Connection?

Представляє з’єднання та керує транзакціями.

17. Для чого використовується Cursor?

Виконує SQL і дозволяє отримувати результати запитів.

18. Для чого потрібен commit()?

Підтверджує зміни поточної транзакції.

19. Для чого потрібен rollback()?

Скасовує непідтверджені зміни транзакції.

20. Що таке SQL injection?

Підміна структури SQL за рахунок вставлення неконтрольованих даних у текст запиту.

21. Що таке parameterized query?

Запит, у якому значення передаються драйверу окремо від SQL через placeholders.

22. Чому не рекомендується формувати SQL через f-string?

Вставлені значення можуть змінити синтаксис запиту; параметри відокремлюють дані від SQL-коду.

23. Що таке Primary Key?

Унікальний ідентифікатор рядка таблиці.

24. Що таке Foreign Key?

Посилання на ключ іншої таблиці, яке підтримує зв’язки та цілісність.

25. Що таке транзакція?

Група операцій, яка повинна завершитися як одна узгоджена одиниця.

26. Що означає Atomicity?

Або підтверджуються всі зміни транзакції, або жодна.

27. Що означає Consistency?

Після коректної транзакції зберігаються визначені обмеження й інваріанти.

28. Що означає Isolation?

Паралельні транзакції взаємодіють за визначеними правилами видимості та блокування.

29. Що означає Durability?

Підтверджені зміни зберігаються відповідно до гарантій сховища та його налаштувань.

30. Що означає ACID?

Atomicity, Consistency, Isolation, Durability — атомарність, узгодженість, ізоляція, довговічність.

31. Що таке persistence layer?

Шар збереження та відновлення даних, відокремлений від інтерфейсу й бізнес-логіки.

32. Для чого потрібний Repository Pattern?

Для приховування деталей запитів за операціями над сутностями.

33. Чому Repository відокремлюють від Service Layer?

Щоб бізнес-сценарії не залежали від конкретного SQL, драйвера або схеми збереження.

34. Що таке ORM?

Object-Relational Mapping — відображення між об’єктами й реляційними таблицями.

35. Для чого використовується SQLAlchemy?

Для роботи з SQL та ORM, сесіями, транзакціями й різними драйверами БД.

36. Що таке ORM model?

Клас, поля й екземпляри якого зіставлено з таблицею та її рядками.

37. Що таке relationship()?

Описує навігаційний зв’язок між ORM-моделями; зовнішній ключ задається окремо.

38. Що означає one-to-many?

Одному рядку відповідає багато пов’язаних рядків, наприклад виробнику — пристрої.

39. Що означає many-to-many?

Багато рядків обох таблиць можуть бути взаємно пов’язані через проміжну таблицю.

40. Що таке migration?

Версійована зміна схеми даних із контрольованим застосуванням.

41. Для чого використовується Alembic?

Для керування міграціями SQLAlchemy-проєктів, зокрема upgrade та downgrade.

42. Для чого потрібна команда alembic upgrade head?

Застосовує всі потрібні міграції до актуальної головної ревізії.

43. Що таке integration test?

Тест взаємодії реальних компонентів системи, наприклад ORM та SQLite.

44. Чим integration test відрізняється від unit test?

Він перевіряє зв’язки й інфраструктуру, а unit test — вузьку поведінку компонента.

45. Навіщо використовувати окрему test database?

Щоб тести не псували робочі дані та мали відомий початковий стан.

46. Що перевіряє тест rollback?

Що після помилки жодна частина пакетної операції не залишилася підтвердженою.

47. Для чого використовуються fixtures?

Для створення та очищення незалежної тестової бази й інших передумов.

48. Що робить SQL JOIN?

Поєднує рядки таблиць за умовою зв’язку.

49. Що робить GROUP BY?

Формує групи рядків для обчислення агрегатів за ключами.

50. Для чого використовуються COUNT, SUM, AVG, MIN, MAX?

COUNT рахує, SUM підсумовує, AVG усереднює, MIN і MAX знаходять крайні значення.

10. Висновок

Додано SQLite з двома пов’язаними таблицями, параметризований DB-API, ORM-репозиторії та цілісну пакетну транзакцію. Міграція location демонструє розвиток схеми без зміни предметної області. Integration tests підтвердили CRUD, обмеження й rollback.